

Sistema de Gestión de Inventario RSM

Informe actualizado: microservicios + agente funcional de IA

Campo	Información actualizada
Asignatura	INGENIERÍA DE SOLUCIONES CON INTELIGENCIA ARTIFICIAL_001D
Evaluación	Evaluación 2
Docente	HECTOR ANDRES MOREL BRIONES
Nombre de equipo	Joaquin Alvarado
Repositorio GitHub	https://github.com/elgatomaleante12/rsm-agente-ep2-entrega

1. Problemática y contexto

Los talleres mecánicos pequeños suelen operar con cuadernos, carpetas físicas y registros en papel. Esto genera fallos frecuentes en el control de stock, errores en pedidos de repuestos, baja trazabilidad de órdenes de trabajo y poca transparencia para el cliente sobre el estado de su vehículo.

El impacto operativo se observa en retrasos de reparación, duplicación de tareas, pérdida de seguimiento y tiempos muertos al buscar disponibilidad de repuestos. El sistema RSM responde a este problema mediante digitalización de procesos, arquitectura de microservicios y un agente funcional de IA para consultar, razonar y registrar información del inventario.

2. Visión del proyecto

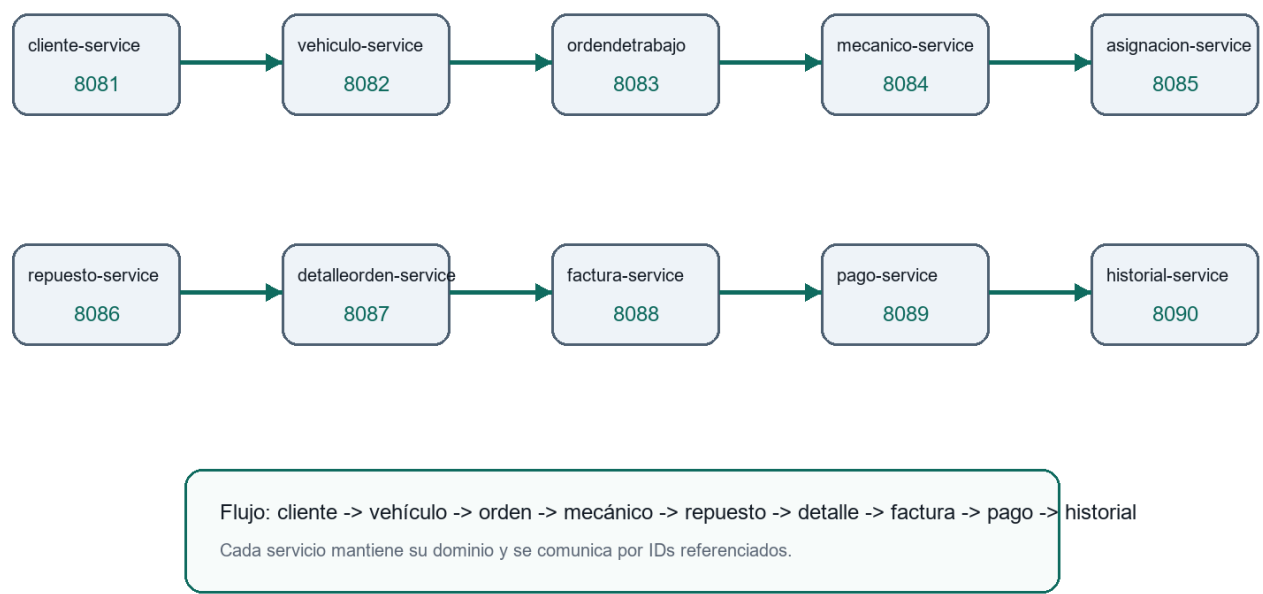
RSM busca transformar la gestión de talleres mecánicos mediante la digitalización completa del flujo operativo. La solución considera sincronización en tiempo real, historial por patente, estados actualizados y trazabilidad desde el ingreso del cliente hasta el cierre de la orden.

- Sincronización de datos entre servicios mediante IDs referenciados.
- Historial técnico por vehículo para mantener continuidad operacional.
- Control de repuestos, costos, facturación y pagos en servicios separados.
- Agente de IA para consultas de stock, alertas, reportes y registro de salidas.

3. Arquitectura de microservicios

El sistema implementa 10 microservicios independientes. Cada servicio cubre una parte del flujo del taller y puede evolucionar de forma modular sin afectar directamente a los demás componentes.

Mapa de servicios RSM



Puerto	Microservicio	Responsabilidad
8081	cliente-service	Registra clientes y datos de contacto.
8082	vehiculo-service	Asocia vehículos con clientes mediante clienteld.
8083	ordendetrabajo-service	Crea órdenes con vehículo, mecánico, estado y descripción.
8084	mecanico-service	Administra mecánicos, especialidad y contacto.
8085	asignacion-service	Asigna mecánicos a vehículos o trabajos del taller.
8086	repuesto-service	Controla repuestos, precios y stock disponible.
8087	detalleorden-service	Registra costos, cantidades y detalles de cada orden.
8088	factura-service	Genera facturas vinculadas a una orden de trabajo.
8089	pago-service	Registra pagos asociados a facturas.
8090	historial-service	Guarda historial técnico de cada vehículo.

4. Modelado estructural y flujo de negocio

Cada microservicio mantiene su propio dominio de datos. El modelado usa entidades relacionadas por IDs referenciados, evitando acoplamiento entre bases de datos y permitiendo despliegues independientes. Las migraciones de esquema se controlan con Flyway y la persistencia se implementa con Spring Data JPA.

El flujo de negocio conecta nueve pasos: registro del cliente, asociación del vehículo, creación de orden, asignación de mecánico, gestión de repuestos, cálculo de detalle y costos, generación de factura, registro de pago y actualización del historial del vehículo.

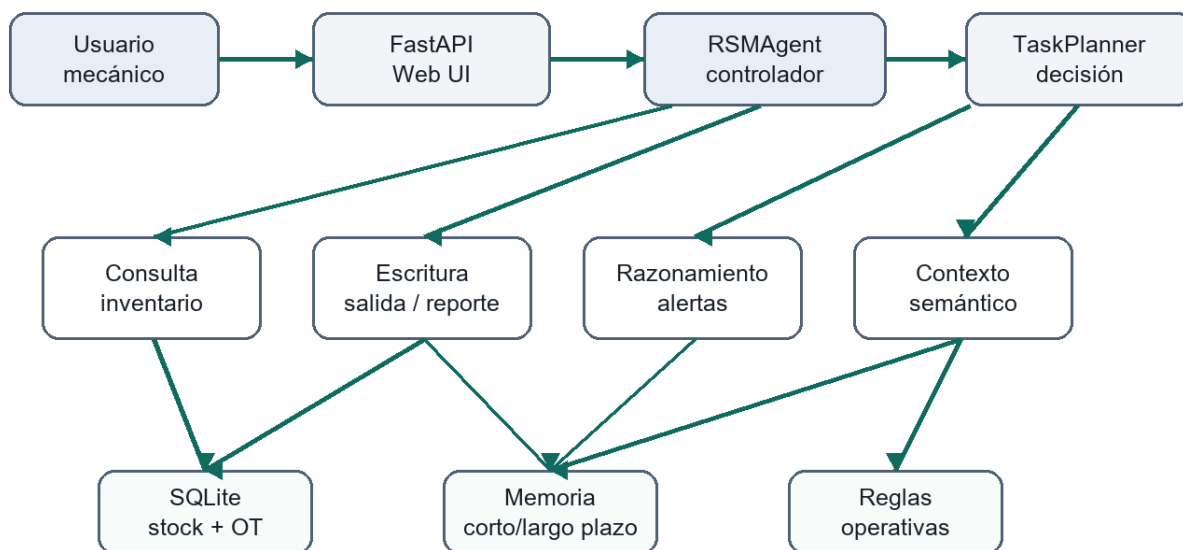
Capa	Tecnología	Rol
Backend microservicios	Java 21 + Spring Boot 3.x	APIs REST para cada dominio del taller.
Persistencia	Spring Data JPA + Hibernate ORM	Mapeo objeto-relacional y repositorios.
Base de datos	MariaDB puerto 5506	Almacenamiento transaccional por servicio.
Migraciones	Flyway	Versionado automático de esquemas.
Validación	Bean Validation	Control de reglas y datos de entrada.
Construcción	Maven	Gestión de dependencias y ciclo de build.

5. Agente funcional de IA agregado

La versión actual incorpora un agente funcional ejecutable en Visual Studio Code. Este agente no reemplaza la arquitectura de microservicios; la complementa como capa inteligente para consulta, escritura, recuperación de contexto y toma de decisiones sobre inventario.

Dato	Detalle
Proyecto	RSM - Agente Funcional EP2 para inventario mecánico.
Repositorio GitHub	https://github.com/elgatomaleante12/rsm-agente-ep2-entrega
Ejecución local	Abrir rsm-agente-ep2.code-workspace y ejecutar las tareas de VS Code.
API	FastAPI disponible en http://127.0.0.1:8000 y documentación en /docs.
Pruebas	pytest validado con resultado: 6 passed.

Orquestación del agente RSM EP2



Evidencia: el agente devuelve plan, herramientas usadas, memoria reciente y respuesta con fuente.

6. Herramientas, memoria y planificación del agente

El agente se implementa en Python con FastAPI, SQLite y LangChain Core. La clase RSMAgent orquesta un planificador determinístico y cinco herramientas: consultar inventario, registrar salida, generar alertas, recuperar contexto y redactar reportes operativos.

- Consulta: recupera stock, ubicación, proveedor y compatibilidad de repuestos.
- Escritura: registra salidas de inventario con orden de trabajo, mecánico y vehículo.
- Razonamiento: clasifica stock CRÍTICO o BAJO y sugiere reposición.
- Memoria: guarda eventos de usuario, respuestas, movimientos y reportes en SQLite.
- Recuperación de contexto: combina inventario, reglas operativas y memoria previa.

La planificación ajusta el comportamiento ante condiciones cambiantes. Si faltan datos de trazabilidad, el agente no descuenta stock; si la cantidad solicitada supera la disponibilidad, rechaza el movimiento; si el stock queda bajo mínimo, registra el evento y advierte la necesidad de reposición.

7. Evidencia de funcionamiento

Escenario	Entrada	Resultado
Consulta de stock	¿Cuántos filtros de aceite Toyota quedan?	Responde FO-TOY-001 con stock, ubicación y proveedor.
Alertas	¿Qué ítems están críticos o bajo mínimo?	Lista ítems CRÍTICO/BAJO con proveedor sugerido.
Salida válida	Salida de 2 unidades FO-TOY-001 para OT-778	Descuenta stock y guarda memoria de trazabilidad.

Salida incompleta	Registra salida de 2 unidades FO-TOY-001	Bloquea escritura por falta de OT, mecánico o vehículo.
Reporte	Genera un reporte de alertas de stock	Crea archivo Markdown en outputs/.

8. Conclusión

El sistema RSM queda documentado como una solución integral: por un lado, presenta una arquitectura fullstack basada en microservicios para digitalizar el flujo completo del taller; por otro, incorpora un agente de IA funcional que permite consultar inventario, tomar decisiones operativas, registrar movimientos y conservar memoria de contexto. Esta combinación responde al requerimiento de construir una solución real, ejecutable, verificable y alineada con Ingeniería de Soluciones con Inteligencia Artificial.

Referencias

FastAPI. (2024). FastAPI documentation. <https://fastapi.tiangolo.com/>

LangChain. (2024). LangChain documentation. <https://python.langchain.com/docs/>

Oracle. (2024). Java documentation. <https://docs.oracle.com/en/java/>

Spring. (2024). Spring Boot documentation. <https://spring.io/projects/spring-boot>

SQLite. (2024). SQLite documentation. <https://sqlite.org/docs.html>